Manuel Utilisateur

Équipe GL36

Ensimag - Projet Génie Logiciel

19 janvier 2026

Résumé

Ce manuel présente le mode opératoire du compilateur `decac`, conçu pour les utilisateurs ayant une connaissance des spécifications du langage Deca, qui souhaitent l'utiliser avec la bibliothèque `Math.h`. Il détaille les étapes d'installation, d'utilisation, ainsi que les limitations actuelles du compilateur et de l'extension.

Table des matières

1 Limitations de l'implémentation **2**

1.1 Fonctionnalités implémentées 2

1.2 Limitations techniques connues 2

2 Gestion des messages d'erreur **2**

2.1 Formatage des erreurs de compilation 2

2.2 Erreurs de lexicographie (Lexer) 2

2.3 Erreurs de syntaxe hors-contexte (Parser) 3

2.4 Erreurs de syntaxe contextuelle 3

2.5 Erreurs d'exécution (Runtime) 3

3 Extension de la Bibliothèque Standard **4**

4 Mode opératoire **4**

4.1 Installation et Prérequis 4

4.2 Utilisation de l'extension Math 4

5 Limitations de l'extension **5**

1 Limitations de l'implémentation

1.1 Fonctionnalités implémentées

Le compilateur implémenté couvre les aspects suivants du langage Deca :

→ Fonctions de base : types élémentaires (`int`, `float`, `boolean`), structures de contrôle (`if`, `while`), entrées (`readInt`, `readFloat`) et sorties (`print`, `println` ainsi que le format hexadécimal `printx` et `printlnx`).

→ Classes et objets : déclaration de classes, héritage, polymorphisme et accès aux champs (`protected` et visibilité publique par défaut).

→ Gestion de la mémoire : allocation dynamique sur le tas via l'instruction `new`.

1.2 Limitations techniques connues

Au travers de nos tests, nous avons identifié une limitation technique majeure dans notre implémentation actuelle du compilateur Deca.

> Un appel à `equals` qu'il s'agisse de la méthode par défaut ou d'une redéfinition utilisateur provoque un arrêt de l'exécution de la machine IMA.

En interne, notre compilateur gère les adresses des méthodes via des offsets calculés par rapport à une base en mémoire. Lors d'un appel, notre génération de code tente d'effectuer un saut vers la méthode en utilisant cet offset directement avec l'instruction `BSR`. Il s'avère cependant que l'opérande ainsi calculé n'est pas une adresse valide pour cette instruction spécifique : IMA n'autorise pour le `BSR` qu'un label ou un registre pur, et rejette tout adressage indirect avec déplacement (`n(R)`).

Par conséquent, lorsqu'un utilisateur appelle la méthode `equals(...)`, le comportement est déterministe mais fatal : la machine IMA lève systématiquement l'erreur `BSR avec operande : adresse memoire`. En l'état, les seules comparaisons d'égalité fonctionnelles sont celles utilisant l'opérateur de comparaison d'adresses `==`.

2 Gestion des messages d'erreur

2.1 Formatage des erreurs de compilation

Toutes les erreurs détectées avant la génération de code (lexicographie, syntaxe et vérification contextuelle) suivent le formatage requis pour l'évaluation automatique :

```
$ <nom_de_fichier.deca>:<ligne>:<colonne>: <description_de_l_erreur>
```

2.2 Erreurs de lexicographie (Lexer)

L'analyseur lexical se charge de reconnaître les jetons (`tokens`). Les erreurs surviennent lorsqu'une séquence ne correspond à aucune règle définie. Ces erreurs sont capturées par le `DecacErrorListener` ou de rattrapées avant dans le `DecaLexer` lui-même (pour `INVALID_IDENT` et `UNTERMINATED_COMMENT`) pour fournir des messages plus clairs :

- `INVALID_IDENT` : Levé par un identificateur qui ne respecte pas les règles lexicographiques (ex : commence par un chiffre).
- `UNTERMINATED_COMMENT` : Levé lorsqu'un commentaire multi-ligne (`/*`) est ouvert mais jamais refermé avant la fin du fichier.
- `token recognition error` : Levé lors de l'utilisation de caractères n'appartenant pas au langage Deca (ex : `#`, `@`) ou d'une chaîne de caractères mal formée (ex : guillemet manquant).

2.3 Erreurs de syntaxe hors-contexte (Parser)

L'analyseur syntaxique vérifie si l'ordre des jetons respecte la grammaire du langage. Sous chaque explication d'erreur est fourni un exemple de fichier test qui lève l'erreur. Ces erreurs sont capturées par le `DecacErrorListener` :

- `mismatched input` : Un jeton a été trouvé là où un autre était attendu (ex : condition de `if` vide).
  Exemple de fichier test : `.../syntax/invalid/parser/empty-if.deca`
- `no viable alternative` : Structure de code incomplète ou ambiguë (ex : point-virgule manquant).
  Exemple de fichier test : `.../syntax/invalid/parser/missing-semi.deca`
- `missing <token>` : Absence d'un symbole obligatoire (ex : parenthèse fermante oubliée).
  Exemple de fichier test : `.../syntax/invalid/parser/missing-token.deca`
- `extraneous input` : Symbole invalide pour la règle en cours (ex : opérateur doublé).
  Exemple de fichier test : `.../syntax/invalid/parser/duplicate-op.deca`

2.4 Erreurs de syntaxe contextuelle

Ces erreurs lors de la décoration de l'AST après qu'il soit construit. Elles concernent la cohérence des types, des variables et des déclarations.

Catégorie	Messages d'erreur et causes possibles
Identificateurs	`Identifier <name> not found` : variable ou méthode non définie. `Type <name> is already defined` : double déclaration d'un type.
Classes	`Superclass <name> of class <C> is not defined` : héritage d'une classe inexistante. `New can only be used with class types` : instanciation sur un type primitif.
Méthodes/Champs	`Field name collides with an inherited method` : conflit lors de l'héritage. `Visibility error for protected field` : accès illégal à un champ protégé.
Types/Opérations	`Operand of UnaryMinus must be of numeric type` : - appliqué sur un booléen. `Condition must be a boolean. Found: <type>` : type invalide dans un `if` ou `while`. `Cannot cast from <T1> to <T2>` : cast illégal entre types incompatibles.
Arguments	`Expected <n> args, got <m>` : signature de méthode non respectée.

2.5 Erreurs d'exécution (Runtime)

Même si le programme est valide contextuellement, il peut encore échouer lors de l'exécution sur la machine virtuelle IMA (règles sémantiques). Ces erreurs sont gérées par notre `RuntimeErrorManager`. Chaque erreur d'exécution est liée à un label spécifique en assembleur. Lorsqu'une condition d'erreur est détectée (avec l'instruction `BOV` par exemple), le programme passe à l'instruction du label ce qui affiche le message et stoppe l'exécution (`ERROR`).

Message d'erreur	Cause(s) possible(s) de l'erreur
`Division by zero`	Tentative de division par un entier ou flottant nul.
`Stack overflow`	Débordement de la pile (appels récursifs trop profonds ou registres).
`Input/Output error`	Erreur lors de la lecture d'un entier ou d'un flottant
`Overflow arith.`	Débordement lors d'une opération sur les nombres flottants.
`Null dereference`	Accès à un champ ou une méthode sur un objet non instancié.
`Heap overflow`	Mémoire globale saturée lors d'une allocation par `new`.
`Impossible cast`	Échec d'un transtypage dynamique entre deux classes incompatibles.

3 Extension de la Bibliothèque Standard

Le compilateur intègre une extension permettant l'utilisation de fonctions mathématiques natives via la classe `Math` (ou via inclusion d'un fichier header `Math.decah`).

- Fonctions trigonométriques : `sin`, `cos`, `asin`, `acos`, `atan`.
- Fonction ulp : `ulp`.

4 Mode opératoire

4.1 Installation et Prérequis

Pour utiliser le compilateur `decac`, votre environnement de travail doit disposer des outils suivants :

→ Java (version 17 ou supérieure).
→ Maven (pour la gestion des dépendances et la construction).
→ La machine IMA installée et ajoutée dans votre `PATH`.

Avant votre première utilisation, compilez le projet à la racine :

```
$ mvn package
```

Le script `decac` est alors généré dans le répertoire `src/main/bin/`. Pour afficher toutes les options disponibles, exécutez simplement la commande `decac` sans arguments.

4.2 Utilisation de l'extension Math

L'extension mathématique vous permet d'accéder aux fonctions trigonométriques précisées dans la partie précédente. Pour les utiliser, incluez le fichier d'en-tête dédié au début de votre fichier source `.deca` :

```
$ #include "Math.decah"
```

Le compilateur résout automatiquement l'inclusion et lie les méthodes nécessaires lors de la génération du code assembleur. Compilez votre fichier utilisant ces fonctions avec la commande :

```
$ decac <fichier.deca>
```

Puis, pour exécuter le code assembleur généré sur la machine IMA, utilisez :

```
$ ima <fichier.ass>
```

5 Limitations de l'extension

L'extension mathématique est soumise à des contraintes de précision liées aux séries de Taylor utilisées en interne et à la précision limitée sur 32 bits :

- Les fonctions trigonométriques garantissent une précision de moins d'un ulp pour des valeurs d'entrée standards. Les erreurs s'accumulent pour des valeurs tendant vers l'infini.
- Un appel à `asin` avec un paramètre hors de son domainde de définition ($[-1, 1]$) retournera `NaN`.